



Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

A04 - Algoritmo Genético Ordinal

Problema del Vendedor Viajero (20 ciudades)

Algoritmos Metaheurísticos

Alumno: José Luis Chávez Gómez

28 de septiembre de 2026

1. Introducción a los Algoritmos Genéticos

Los algoritmos genéticos (AG) son una familia de métodos de optimización inspirados en la evolución biológica por selección natural, propuestos formalmente por John Holland en la década de 1970 [1] y popularizados como herramienta de búsqueda y optimización por Goldberg [2]. En lugar de mejorar una única solución candidata, como lo hacen el gradiente descendente o el Hill Climbing revisados en la actividad anterior, un AG mantiene una **población** de soluciones que evoluciona a lo largo de sucesivas **generaciones**, de modo que las soluciones de mejor calidad tienen mayor probabilidad de transmitir sus características a la siguiente generación [3, 4].

Cada solución candidata se representa mediante un **cromosoma**, formado por **genes** que codifican las variables del problema. La calidad de un cromosoma se mide con una función de **aptitud** (*fitness*), que en un problema de minimización como el que se aborda en este reporte corresponde directamente a la función de costo. El ciclo básico de un AG consta de los siguientes pasos:

1. **Inicialización:** se genera una población inicial, normalmente de forma aleatoria.
2. **Evaluación:** se calcula la aptitud de cada cromosoma.
3. **Selección:** se eligen los individuos que actuarán como padres, favoreciendo a los de mejor aptitud (por ejemplo, mediante torneos o ruleta).
4. **Reproducción:** se generan hijos aplicando operadores genéticos, principalmente el *cruzamiento* (combinación de material genético de dos padres) y la *mutación* (alteración aleatoria de uno o más genes).
5. **Reemplazo:** los hijos forman la nueva población, en ocasiones conservando intacto al mejor individuo (*elitismo*).
6. **Terminación:** el proceso se repite hasta cumplir un criterio de paro, como un número máximo de generaciones.

La eficacia de un AG proviene del equilibrio entre **exploración** (visitar regiones diversas del espacio de búsqueda, función principal de la población y de la mutación) y **explotación** (refinar las mejores soluciones encontradas, función de la selección). Este equilibrio es justamente la limitación que se observó en los métodos de búsqueda directa de la actividad A03: los métodos puramente exploratorios refinan poco su solución, mientras que los puramente locales quedan atrapados en el primer óptimo local cercano.

La forma de codificar el cromosoma depende del problema; las representaciones más comunes son la binaria, la de números reales y la de **permutaciones**. En este último caso, que es el de los problemas de ordenamiento como el vendedor viajero, cada cromosoma es una permutación de n elementos y lo que importa es el *orden* de los genes. A este tipo de AG se le denomina en el curso **algoritmo genético ordinal** (o basado en el orden). Su principal particularidad es que los operadores clásicos de cruzamiento y mutación, diseñados para cadenas binarias o reales, suelen producir cromosomas inválidos (con ciudades repetidas u omitidas), por lo que se requieren operadores que siempre preserven la validez de la permutación [5]. Las mutaciones por *reversa* y por *enroque* utilizadas en esta actividad cumplen esta condición por construcción.

2. El Problema del Vendedor Viajero

El problema del vendedor viajero (*Traveling Salesman Problem*, TSP) consiste en encontrar el recorrido de menor longitud que visita exactamente una vez cada una de n ciudades y regresa a la ciudad de partida. Formalmente, dadas las posiciones $c_1, \dots, c_n \in \mathbb{R}^2$ de las ciudades, se busca la permutación π de $\{1, \dots, n\}$ que minimiza

$$f(\pi) = \sum_{i=1}^{n-1} \|c_{\pi(i)} - c_{\pi(i+1)}\| + \|c_{\pi(n)} - c_{\pi(1)}\| \quad (1)$$

donde $\|\cdot\|$ es la distancia euclidiana. A pesar de su sencilla formulación, el TSP es un problema de optimización combinatoria clasificado como NP-difícil [7]: para n ciudades y distancias simétricas existen $(n-1)!/2$ recorridos distintos, lo que para las $n = 20$ ciudades de esta actividad equivale a $19!/2 \approx 6,08 \times 10^{16}$ rutas. Una búsqueda exhaustiva como la de la actividad A03 es, por tanto, completamente inviable, y este es el tipo de problema en el que los metaheurísticos resultan valiosos: no garantizan el óptimo, pero encuentran soluciones de buena calidad con un número de evaluaciones muy inferior. El TSP es además un problema de referencia por sus aplicaciones prácticas, como la logística y planeación de rutas, la perforación de circuitos impresos o la secuenciación de tareas [8].

3. Algoritmo Genético Ordinal Implementado

3.1. Representación y función objetivo

Cada cromosoma es una lista con una permutación de los índices $0, 1, \dots, 19$, donde la posición en la lista indica el orden de visita. Las ciudades se generan con coordenadas aleatorias uniformes en el cuadro $[-100, 100] \times [-100, 100]$. La aptitud de un cromosoma es la longitud total del recorrido cerrado, ecuación (1), por lo que se busca **minimizarla**.

3.2. Selección por torneo

Para generar cada hijo se elige un padre mediante un torneo: se toman al azar k individuos de la población y gana el de menor distancia total. El tamaño del torneo se define como un porcentaje de la población (5%, es decir $k = 5$ con 100 individuos), lo que controla la presión selectiva: un torneo pequeño favorece la diversidad, mientras que uno grande acelera la convergencia pero aumenta el riesgo de convergencia prematura [6].

3.3. Operadores de reproducción

Este algoritmo no utiliza cruzamiento; cada hijo se obtiene aplicando una mutación al padre ganador del torneo. Con probabilidad 0,5 se aplica cada uno de los siguientes procesos:

- **Reversa:** se eligen dos posiciones aleatorias $i < j$ y se invierte el orden de los genes comprendidos entre ambas. En el TSP euclidiano equivale a un movimiento de tipo 2-opt, que elimina cruces entre aristas del recorrido.
- **Enroque:** se extrae el gen de la posición i y se inserta en la posición j , desplazando los genes intermedios. Reubica una sola ciudad dentro del recorrido.

Ambos operadores producen siempre una permutación válida, por lo que no es necesario reparar a los hijos.

3.4. Elitismo y registro del mejor global

En cada generación el mejor cromosoma pasa sin modificarse a la siguiente población (elitismo), de modo que la mejor distancia nunca empeora. Además, se almacena la mejor ruta global encontrada en todo el proceso, de forma que la solución final reportada corresponda siempre a la distancia mínima registrada en el historial.

3.5. Parámetros utilizados

Parámetro	Valor
Número de ciudades	20
Tamaño de la población	100
Número de generaciones	500
Tamaño del torneo	5 (5 % de la población)
Operadores de mutación	Reversa y enroque (50 % cada uno)
Cruzamiento	No se utiliza
Elitismo	1 individuo
Semilla aleatoria	42
Evaluaciones de la función objetivo	$\approx 50,000$

Cuadro 1: Parámetros del algoritmo genético ordinal.

4. Código Fuente

```
1  """ Bibliotecas necesarias
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import random
5
6  """ Parámetros del problema y del algoritmo
7  NUM_CITIES = 20                # número de ciudades del problema
8  POP_SIZE = 100                # tamaño de la población
9  GENERATIONS = 500             # número de generaciones
10 TOURNAMENT_PERCENT = 0.05      # porcentaje de la población que compete en cada
    torneo
11 TOURNAMENT_SIZE = max(2, int(POP_SIZE * TOURNAMENT_PERCENT))
12 ELITISM = True                # si es True, el mejor cromosoma pasa intacto a
    la siguiente generación
13 SEED = 42                     # semilla para poder reproducir los resultados (
    None = aleatorio)
14
15 if SEED is not None:
16     random.seed(SEED)
17     np.random.seed(SEED)
18
19 # Posición aleatoria de las ciudades en el cuadro [-100, 100] x [-100, 100]
20 CITIES = -100 + 200 * np.random.rand(NUM_CITIES, 2)
21
22 """ Función objetivo
23 def distancia_ciudades(ciudad1, ciudad2):
24     """Distancia euclidiana entre dos ciudades."""
25     return np.linalg.norm(ciudad1 - ciudad2)
26
27 def distancia_total(ruta):
28     """
29     Longitud total de una ruta (ciclo cerrado): suma las distancias entre
30     ciudades consecutivas y regresa de la última a la primera.
31     """
32     distancia = 0
33     for i in range(NUM_CITIES):
34         ciudad_actual = CITIES[ruta[i]]
35         ciudad_siguiente = CITIES[ruta[(i + 1) % NUM_CITIES]]
36         distancia += distancia_ciudades(ciudad_actual, ciudad_siguiente)
37     return distancia
38
```

```

39 %% Funciones de ayuda del algoritmo genético ordinal
40 def crear_cromosoma():
41     """Un cromosoma es una permutación de las ciudades (representación ordinal).
42         """
43     return random.sample(range(NUM_CITIES), NUM_CITIES)
44
45 def crear_poblacion():
46     """Población inicial de POP_SIZE rutas aleatorias."""
47     return [crear_cromosoma() for _ in range(POP_SIZE)]
48
49 %% Procesos reproductivos (mutaciones que siempre producen permutaciones válidas)
50 def proceso_reversa(padre):
51     """Invierte el orden de los genes entre dos posiciones aleatorias i y j."""
52     hijo = padre.copy()
53     if len(hijo) > 2:
54         i, j = sorted(random.sample(range(len(hijo)), 2))
55         hijo[i:j+1] = reversed(hijo[i:j+1])
56     return hijo
57
58 def proceso_enroque(padre):
59     """Extrae el gen de la posición i y lo inserta en la posición j."""
60     hijo = padre.copy()
61     if len(hijo) > 2:
62         i, j = sorted(random.sample(range(len(hijo)), 2))
63         gen = hijo.pop(i)
64         hijo.insert(j, gen)
65     return hijo
66
67 def torneo(poblacion, distancias):
68     """Selección por torneo: gana la ruta más corta de TOURNAMENT_SIZE elegidas
69         al azar."""
70     participantes_idx = random.sample(range(POP_SIZE), TOURNAMENT_SIZE)
71     distancias_participantes = [distancias[i] for i in participantes_idx]
72     return poblacion[participantes_idx[int(np.argmin(distancias_participantes))]]
73
74 %% Algoritmo genético ordinal
75 def algoritmo_genetico():
76     """
77     Regresa:
78     mejor_ruta_global : mejor cromosoma encontrado en TODAS las generaciones
79     mejor_dist_global : su distancia total
80     hist_mejor        : mejor distancia de cada generación
81     hist_promedio     : distancia promedio de cada generación
82     ruta_inicial      : mejor ruta de la población inicial (para comparar)
83     """
84     poblacion = crear_poblacion()
85
86     hist_mejor = []
87     hist_promedio = []
88     mejor_ruta_global = None
89     mejor_dist_global = np.inf
90     ruta_inicial = None
91
92     for g in range(GENERATIONS):
93         # 1. Evaluar toda la población
94         distancias = [distancia_total(cromosoma) for cromosoma in poblacion]
95
96         # 2. Mejor de la generación
97         mejor_idx = int(np.argmin(distancias))
98         mejor_cromosoma = poblacion[mejor_idx].copy()
99         mejor_distancia = distancias[mejor_idx]

```

```

98     hist_mejor.append(mejor_distancia)
99     hist_promedio.append(float(np.mean(distancias)))
100
101
102     if g == 0:
103         ruta_inicial = mejor_cromosoma.copy()
104
105     # 3. Actualizar el mejor global (evita perder la mejor ruta si no hay
106         elitismo)
107     if mejor_distancia < mejor_dist_global:
108         mejor_dist_global = mejor_distancia
109         mejor_ruta_global = mejor_cromosoma.copy()
110
111     # 4. Generar los hijos: selección por torneo + mutación
112     hijos = []
113     for _ in range(POP_SIZE):
114         ganador = torneo(poblacion, distancias)
115
116         # Se elige al azar el proceso reproductivo
117         if random.random() < 0.5:
118             hijo = proceso_reversa(ganador)
119         else:
120             hijo = proceso_enroque(ganador)
121         hijos.append(hijo)
122
123     # 5. Elitismo: el mejor de la generación sobrevive sin modificarse
124     if ELITISM:
125         hijos[0] = mejor_cromosoma.copy()
126
127     poblacion = hijos
128
129     if g % 50 == 0:
130         print(f"Generacion {g}: Mejor distancia = {mejor_distancia:.2f}")
131
132     return mejor_ruta_global, mejor_dist_global, hist_mejor, hist_promedio,
133         ruta_inicial
134
135 %% Gráficas
136 def graficar_ruta(ruta, titulo, nombre_archivo):
137     """Dibuja las ciudades y la ruta como ciclo cerrado."""
138     coords = CITIES[ruta + [ruta[0]]] # se agrega la primera ciudad al
139         final para cerrar el ciclo
140     plt.figure(figsize=(7, 7))
141     plt.plot(coords[:, 0], coords[:, 1], 'o-', color='tab:blue')
142     plt.plot(CITIES[ruta[0], 0], CITIES[ruta[0], 1], 's', color='tab:red',
143         markersize=10, label='Ciudad de inicio')
144     for k in range(NUM_CITIES):
145         plt.annotate(str(k), (CITIES[k, 0], CITIES[k, 1]),
146             textcoords="offset points", xytext=(5, 5), fontsize=9)
147     plt.title(titulo)
148     plt.xlabel('x')
149     plt.ylabel('y')
150     plt.legend()
151     plt.grid(alpha=0.3)
152     plt.savefig(nombre_archivo, dpi=200, bbox_inches='tight')
153
154 def graficar_historial(hist_mejor, hist_promedio, nombre_archivo):
155     """Dibuja el historial de la función de costo (distancia total) por generaci
156         ón."""
157     plt.figure(figsize=(8, 5))
158     plt.plot(hist_promedio, label='Promedio de la población', color='tab:orange',
159         alpha=0.8)

```

```

155 plt.plot(hist_mejor, label='Mejor de la generación', color='tab:blue',
156          linewidth=2)
157 plt.xlabel('Generación')
158 plt.ylabel('Distancia total')
159 plt.title('Historial de la función de costo')
160 plt.legend()
161 plt.grid(alpha=0.3)
162 plt.savefig(nombre_archivo, dpi=200, bbox_inches='tight')
163
164 ### Programa principal
165 if __name__ == "__main__":
166     mejor_ruta, mejor_distancia, hist_mejor, hist_promedio, ruta_inicial =
167         algoritmo_genetico()
168
169     print("\nMejor ruta:", mejor_ruta)
170     print(f"Mejor distancia: {mejor_distancia:.2f}")
171     print(f"Mejor distancia de la población inicial: {hist_mejor[0]:.2f}")
172
173     graficar_ruta(ruta_inicial, f"Mejor ruta de la población inicial (distancia
174         = {hist_mejor[0]:.2f})",
175                  "ruta_inicial.png")
176     graficar_ruta(mejor_ruta, f"Mejor ruta encontrada (distancia = {
177         mejor_distancia:.2f})",
178                  "ruta_optima.png")
179     graficar_historial(hist_mejor, hist_promedio, "historial_costo.png")
180     plt.show()

```

5. Resultados

Con la semilla 42, el algoritmo redujo la distancia total del mejor recorrido de 1664,75 (mejor ruta de la población inicial) a 772,86, una reducción de aproximadamente 53,6 %. La Figura 1 muestra la mejor ruta de la población inicial, con múltiples cruces entre aristas, y la Figura 2 muestra la mejor ruta encontrada por el algoritmo.

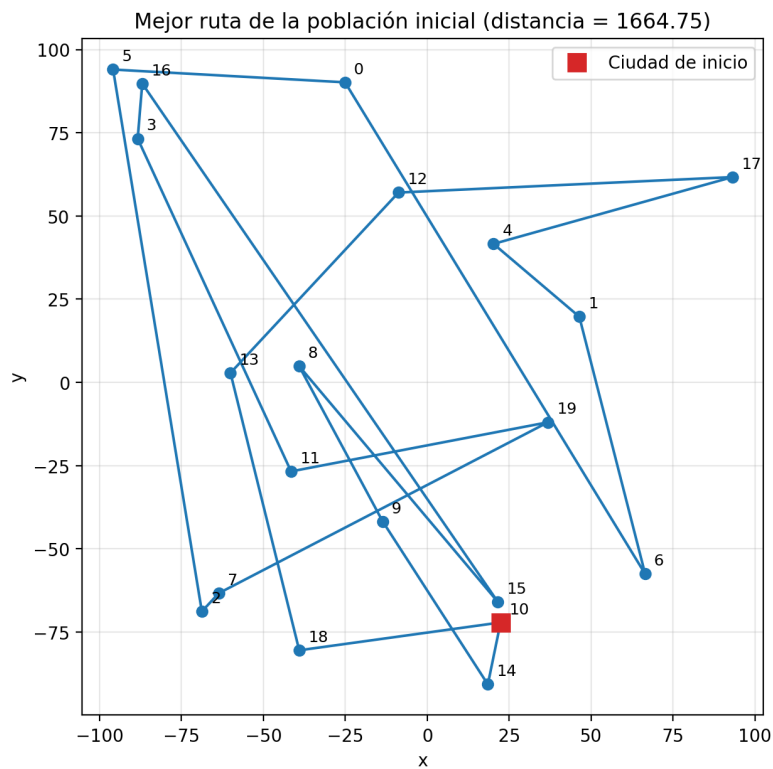


Figura 1: Mejor ruta de la población inicial (distancia total = 1664,75).

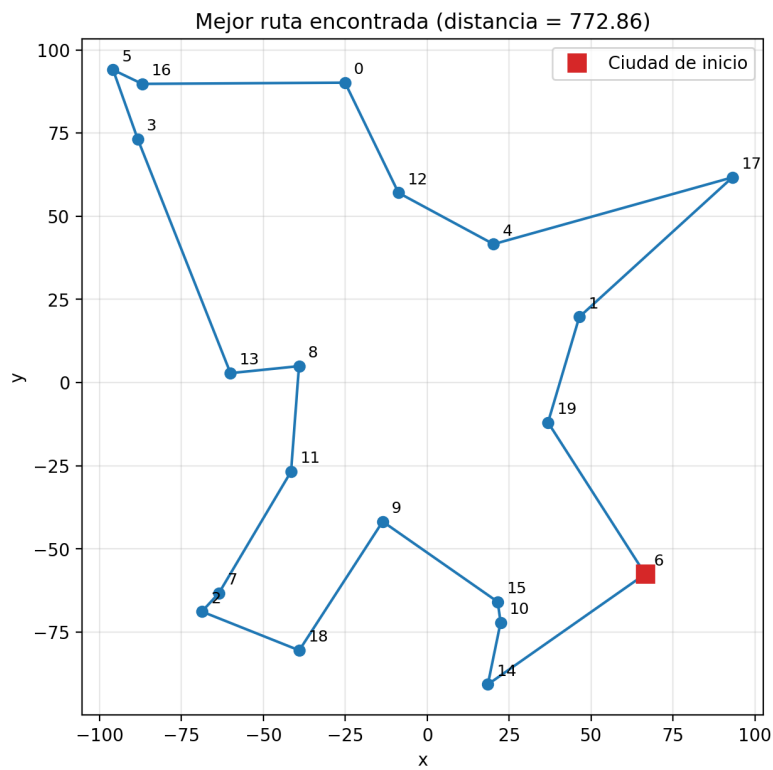


Figura 2: Mejor ruta encontrada por el algoritmo genético (distancia total = 772,86). El cuadro rojo indica la ciudad de inicio.

La Figura 3 presenta el historial de la función de costo. La mejor distancia de cada generación

desciende rápidamente durante las primeras generaciones y alcanza su valor final en la generación 122, tras lo cual permanece constante gracias al elitismo. La distancia promedio de la población desciende de 2160,79 a valores cercanos a 1100, pero no se acerca a la del mejor individuo: como todos los hijos son mutados en cada generación, la población mantiene continuamente una fracción de individuos de menor calidad, lo que conserva su diversidad.

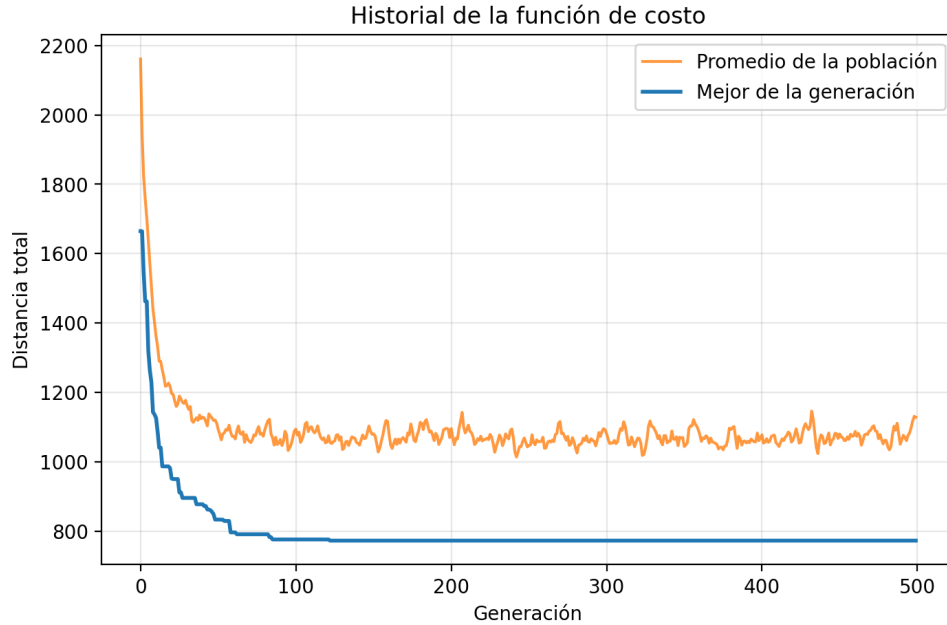


Figura 3: Historial de la función de costo: mejor distancia de cada generación y distancia promedio de la población.

Métrica	Valor
Mejor distancia de la población inicial	1664,75
Mejor distancia encontrada	772,86
Reducción de la distancia	53,6 %
Generación en que se alcanzó la mejor distancia	122
Distancia promedio, generación 0	2160,79
Distancia promedio, generación final	1128,59

Cuadro 2: Resumen de resultados de la corrida con semilla 42.

6. Conclusión

El algoritmo genético ordinal resolvió de forma satisfactoria una instancia del vendedor viajero de 20 ciudades, un problema cuyo espacio de búsqueda contiene del orden de 10^{16} recorridos y que, por tanto, no puede abordarse mediante búsqueda exhaustiva. Con apenas alrededor de 50,000 evaluaciones de la función objetivo, la población pasó de rutas aleatorias llenas de cruces a un recorrido sin cruces visibles entre aristas, con una distancia 53,6 % menor que la del mejor individuo inicial. El historial de costo muestra además el comportamiento típico de un AG con elitismo: una mejora muy rápida al inicio, cuando abundan las mutaciones que corrigen defectos evidentes de la ruta, seguida de una etapa de estancamiento en la que ya solo resultan útiles cambios muy específicos.

Es importante, sin embargo, ser cauteloso con los resultados. Un AG es un método estocástico que no garantiza encontrar el óptimo global, y la ausencia de cruces en la ruta final es una

condición necesaria pero no suficiente de optimalidad. Los resultados provienen de una única corrida con una semilla fija; para una evaluación más rigurosa habría que repetir el experimento con distintas semillas y distintas distribuciones de ciudades, y comparar contra un método de referencia. Además, el hecho de que la mejor distancia se estabilice desde la generación 122 sugiere que la población pierde diversidad útil, y que operadores adicionales, como un cruzamiento que preserve el orden de las ciudades o una búsqueda local posterior, podrían mejorar la solución.

Finalmente, esta actividad permite contrastar el enfoque poblacional con los métodos de la actividad A03. A diferencia del Hill Climbing, que mantiene una sola solución y solo acepta mejoras, el AG conserva una población diversa y selecciona sus padres por torneo, lo que equilibra la exploración del espacio de búsqueda con la explotación de las mejores rutas, uno de los compromisos centrales de los metaheurísticos.

Referencias

- [1] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [2] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- [3] Simon, D. (2013). *Evolutionary Optimization Algorithms: Biologically-Inspired and Population-Based Approaches to Computer Intelligence*. John Wiley & Sons.
- [4] Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer.
- [5] Larrañaga, P., Kuijpers, C. M. H., Murga, R. H., Inza, I., & Dizdarevic, S. (1999). Genetic algorithms for the travelling salesman problem: A review of representations and operators. *Artificial Intelligence Review*, 13, 129–170.
- [6] Miller, B. L., & Goldberg, D. E. (1995). Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, 9(3), 193–212.
- [7] Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [8] Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press.